



Ecosystem Documentation

Technical and Architectural Overview

WORKING VERSION · SEPTEMBER 2026

Preliminary note

This document describes the architecture, current state and technical direction of a decentralised finance ecosystem under development on BNB Smart Chain. It is written for a technically literate reader, and for that reason avoids promotional language. Every component appears with its actual status, every comparative claim rests on verifiable research, and the limitations of the system are described with the same care as its capabilities.

These pages contain no financial projections, no performance promises and no requests. The purpose is that whoever reads it can form their own judgement about what we are building.

1. Summary

We are building a decentralised finance ecosystem in which the entirety of the economic logic resides in smart contracts deployed and verifiable on BNB Smart Chain, and whose application layer never holds custody of user funds, at any point and under any circumstance.

The absence of custody is not, in itself, novel. A considerable number of platforms operate under that model today, and it would be inaccurate to present it as a distinguishing feature. What is unusual, and where the technical interest of this project lies, is the breadth of functions we are bringing together under a single non-custodial layer, with no hybrid or custodial component in between, and the way those modules communicate with one another through an on-chain accounting layer.

The current landscape consists mostly of specialised tools. Some solve asset exchange. Others, derivatives. Others, staking. Others, peer-to-peer commerce. Platforms that bring several of those functions together tend to retain a centralised, custodial core, or they segment the experience between a decentralised mode and one that is not. Our approach starts from a different premise: that a user should be able to exchange assets, automate strategies, trade derivatives, deal directly with another user, put their assets to work, and audit the security of their own wallet, without ever relinquishing control of their funds and without an account that can be suspended.

Four modules are operational today across ten deployed contracts: the exchange layer with its token listing system, the peer-to-peer market, staking, and a complete suite of wallet security tools. The derivatives module has its interface built and its contract under active development. Two further modules are being rebuilt. The project has not been opened to external users and will not be until development reaches a state that warrants it.

It is built by two people. One contributes the engineering and the market expertise; the other, the resources and the relationships. This is a deliberately small team for this phase, and bringing in additional people is anticipated when operations require it, not before.

2. Technical positioning and architectural principles

2.1 What we are building

The most precise category for this system is not that of a decentralised exchange, because an exchange solves asset trading and here trading is one module among several. Nor is it an aggregator, because we do not confine ourselves to routing into third-party protocols. The accurate description is a **non-custodial application layer on which independent but interconnected financial services are articulated**, each backed by its own contracts, and all sharing the same premise: the user signs, the user controls, the user keeps.

2.2 The principles that govern the technical decisions

Four principles explain why the system is built the way it is, and not otherwise.

The first is that the economic logic lives on chain. There is no database recording user balances, because there are no user balances: assets remain in the wallets of their owners. The application layer constructs transactions, presents them to the user, and submits them to the network once signed. Were that layer to disappear tomorrow, the contracts would continue to function and the funds would remain accessible directly from the chain.

The second is modularity with interconnection. Each service has its own contract, with its own bounded responsibility. A failure in one module does not compromise the others, and each can evolve independently. But those modules are not sealed compartments: they communicate through a shared accounting layer that records system activity and feeds the rewards mechanism.

The third is evolution without rupture. All contracts follow an upgradeable proxy pattern. The deployed bytecode is immutable, as befits a contract on a public chain, but the pattern allows the logic to be pointed at an improved implementation while preserving state and address. This carries an implication worth stating plainly, because a technical reader will ask: upgrade authority is restricted to the owner addresses, which constitutes a trust vector we examine without evasion further on. The counterpart is that the system can incorporate new capabilities, correct behaviours and connect to future modules without migrating users or abandoning addresses.

The fourth is that the architecture contains no borders. There are no accounts, no registration, no identity verification and no custody. A system of this kind has no point at which to apply a jurisdictional restriction, simply because there is nothing to restrict: there is no account to suspend and no balance to freeze. This warrants precision, because it is a consequence rather than a stance. Research into the sector shows that even platforms presenting themselves as decentralised apply geographic blocks, and that centralised venues exclude entire jurisdictions, developed countries among them. In our architecture that capability does not exist, not because we chose to forgo it, but because the design does not contain it.

2.3 What is ours and what depends on third parties

Ours is the set of smart contracts, the complete application layer, the logic of each module and the design of the interconnection.

Dependent on external infrastructure are chain access through RPC providers, indexed chain data, price references, and the liquidity of the markets against which trades execute. This dependency is real and we document it in the infrastructure chapter, including what happens when one of those pieces fails, because it has already happened to us.

3. General architecture

3.1 The complete path

It is worth walking through the system in the order a real operation traverses it, because that is how each responsibility becomes clear.

Everything begins at the **user's wallet**, which in this architecture serves three purposes at once: it is their identity, their authentication method and their custody. There is no prior registration and no credential to remember. The wallet connects through the established standards for communication between applications and wallets, and from that moment the application knows a public address and nothing more.

The **application layer** is where the experience takes place. It constructs transactions, calculates trade routes, presents contract state, and translates user intent into concrete calls. What it does not do matters: it has no access to private keys, it cannot initiate movements of funds, and it holds no assets. Its maximum capability is to propose a transaction the user may reject.

The **smart contracts** contain the whole of the economic logic. Each has a delimited responsibility and public functions that anyone can inspect on a chain explorer. When the user signs, the transaction travels directly from their wallet to the network; the application layer is not an intermediary in the movement of value, only in the construction of the instruction.

The **chain** is BNB Smart Chain, chosen for transaction cost, confirmation speed and the depth of its liquidity ecosystem. Being compatible with the Ethereum Virtual Machine, the contracts are written in Solidity and the tooling ecosystem is the industry standard.

Around that core operate the **data sources**: node access providers, indexed data services for historical queries, and price references. All of them feed the interface and none of them intervene in custody.

3.2 The interconnection between modules

This is the point that separates the architecture from a set of independent applications sharing a domain.

Each module, when it generates economic activity, reports it to an **accounting contract** that acts as the system's central register. That contract holds no funds and executes no operations: it accounts. It records which service generated which activity, what proportion corresponds to the rewards mechanism, how many distinct wallets have interacted with the system, and it emits events that allow the full history to be reconstructed from the chain itself.

From there emerges the behaviour that makes the whole act as a system rather than a collection. Activity in the exchange module feeds the rewards mechanism. Activity in the peer-to-peer market does likewise. The security module contributes both to the rewards mechanism and to a fund intended to cover network costs for users who find themselves without the means to transact. Every new module will connect to the same accounting layer, so that the system gains density as it grows: more services means more reported activity, and more activity means more value flowing toward those who sustain the system.

The architectural consequence is that modules do not compete for attention within the same interface; they reinforce one another. The practical consequence is that adding a new service does not require redesigning the system: it requires writing its contract, connecting it to the accounting layer, and authorising it.

4. The non-custodial model, precisely

4.1 What it means here

The term "non-custodial" is used loosely across the sector, so it is worth defining what it means in this particular implementation.

It means that at no point in the course of an operation do user funds pass through an address we control. There is no prior deposit. There is no internal balance. There is no platform hot wallet where user assets are pooled. When a user exchanges an asset, the transaction occurs between their wallet and the corresponding contract; when they run an automated strategy, that strategy acts on bounded permissions the user grants and can revoke; when they participate in the peer-to-peer market, assets move according to conditions encoded in the contract.

It also means that authentication is cryptographic rather than delegated. We do not validate a password or issue a session: we verify that a signature corresponds to an address. There are no credentials that could leak from our side because we store none.

4.2 What it contributes architecturally

The first contribution is the elimination of an entire category of risk. The most substantial historical losses in the sector stem from custodial platforms that were compromised or became insolvent. A system that never holds funds cannot lose them in that manner, nor can it become insolvent toward its users.

The second is continuity. Were our infrastructure to go offline, the contracts would remain deployed and users could interact with them directly. Funds do not depend on our continued existence.

The third is verifiability. All activity is recorded on a public chain. A third party can audit the system's behaviour without asking our permission or trusting our reporting.

The fourth, already noted, is the structural absence of a point at which geographic restriction could be applied.

4.3 What risks remain

It would be dishonest to present the non-custodial model as a complete security solution, and a discerning reader would detect that overstatement immediately. These are the risks the model does not remove.

User error is irreversible. There is no password recovery, no transaction reversal and no support desk that can undo a transfer to the wrong address. The counterpart of autonomy is responsibility.

Granted permissions are a genuine vector. For a contract to move a token on the user's behalf, the user must grant an allowance. That allowance, if unlimited and granted to a malicious contract, permits the asset to be drained. It is among the most widespread loss vectors in the sector, and it is the reason we built a dedicated module for permission analysis and revocation.

Code can contain errors. The contracts are written following established practice and have been reviewed internally, but they have not undergone independent external audit, which belongs to a later phase of development and which we address explicitly in the security chapter.

Upgrade authority concentrates trust. The proxy pattern allows the system to improve, but whoever controls the owner addresses can modify the logic. We describe it for what it is: a conscious decision that prioritises the capacity to evolve during development, with mitigations detailed further on.

Dependence on external infrastructure introduces points of failure that do not compromise funds but do affect service availability.

5. Smart contracts and verifiability

5.1 A distinction worth establishing first

The sector uses as equivalents a set of terms that are not, and precision matters here.

Published code means the source is accessible. **Verified contract** means a chain explorer has confirmed that the source compiles to exactly the deployed bytecode, so that what one reads is what executes. **On-chain transparency** means all contract activity is publicly observable. **Independent audit** means a specialist firm has reviewed the code for vulnerabilities. **Internal review** means the team has reviewed it. And **proven security** is a term that, honestly, no system should claim.

Our contracts are published and verified. They have undergone internal review. They have not been through independent audit, because development continues and an audit of code still evolving would carry limited value. External audit is anticipated in the project's sequence, with the intention of engaging an established firm once development reaches its complete state.

5.2 The deployed contracts

All operate on BNB Smart Chain under an upgradeable proxy pattern. The addresses below are public and can be inspected on any chain explorer.

Accounting · `0x7FdE85E0bD53208F380980cfE317A9D4982434Ab` The system's central register. It receives activity reports from the other modules, accumulates totals by service and by period, counts unique wallets, and emits events that allow the full history to be reconstructed from the chain. It holds no funds and performs

no economic operations.

Price oracle · `0xf51bf11D8C8905bc044B7Fb3B002Bf3F84c977f3` Supplies price references to the modules that require them, allowing amounts to be expressed in stable value even when payment is made in the network's native asset.

Fees · `0x068729CBB708713266FFdE2374e51db2B063FD7C` Centralises fee configuration and the administrative permission checks other contracts consult.

Staking · `0xdC4802d8871cEf57A34e4e0E3b1a87226a4A84C4` Receives the share of system activity allocated to rewarding those who commit assets. It is the point of economic convergence across the modules.

Automated strategies · `0x4e86430BC2260FE359d1Ea7Eef8B595fB241F93B` Holds the logic for the automated strategies that operate on bounded permissions granted by the user.

Token listing · `0x39c48394068299Aa3e3ab114F16bfc3DE11F4112` Governs the mechanism by which a project can bring its token into the system.

Peer-to-peer market · `0x17B47a8Fb97F8980b96c94E4b9137182e0Bf8025` Handles offers, escrowed deposits and the settlement of operations between users.

Market profiles · `0xC01B61B702011747B4c0Ee6B5F2d0F2b4B66880c` Maintains the history and reputation of peer-to-peer market participants.

Wallet security · `0x24E34b95dBd7786b0d11E00e7FA256A8763B6D05` Governs access to the security toolset and distributes the resulting activity across its destinations.

Network cost fund · `0x71763E9Ad60d3D2Baa833496F8b4f8eeD497B65F` Maintains a pool that allows users without the means to cover network costs to obtain the minimum required to move their own assets. It is fed by system activity.

5.3 What a third party can verify independently

A technical reader can, without our cooperation and without trusting anything we assert: read the complete source of each contract; confirm that source corresponds to the deployed bytecode; inspect every public function and its access restrictions; observe every historical transaction; read the emitted events and reconstruct activity; verify the connections between contracts by querying their public variables; and confirm that none of them contains a function permitting the withdrawal of funds from a user wallet.

That final check is the most consequential, and it is the reason we publish the addresses.

6. The modules of the ecosystem

Each module is described with the same structure: what it is, what problem it solves, how the mechanism works, its current state and where it can grow. The status labels are deliberately literal.

6.1 Asset exchange · OPERATIONAL

What it is. An exchange module operating against existing on-chain liquidity, executing from the user's wallet.

What it solves. On-chain exchange typically forces the user to move between interfaces depending on which protocol offers the better price, and to trust that the token being purchased is the one they believe it to be.

How it works. The application layer queries available liquidity, calculates the route and constructs the transaction. The user approves the spend of the source asset where required and signs the operation. Settlement occurs directly between their wallet and the exchange protocol.

Relevant detail. The module currently operates against the most recent generation of the network's exchange protocols. Support for earlier versions is in development, with a concrete practical consequence: it widens the universe of tradeable assets to those whose liquidity sits in earlier-generation pools, which are numerous. The proxy architecture further allows new exchange protocols to be incorporated in future without replacing the contract or migrating anything.

Embedded function: token listing. Any project can bring its token into the system through a mechanism combining verification of the contract address, confirmation that real liquidity exists, and an escrowed deposit that disincentivises fraudulent use. The purpose is for asset onboarding to be open without that meaning the door is open to anything.

Where it grows. Multi-protocol support, route aggregation, and eventually cross-chain operation.

6.2 Automated strategies · OPERATIONAL

What it is. A set of strategies that execute operations automatically according to user-defined parameters, without requiring the user to be present.

What it solves. Strategy automation typically requires depositing funds with a platform that trades on your behalf, which reintroduces precisely the custody risk the user was trying to avoid. Here the strategy operates on bounded permissions, and assets remain in the user's wallet throughout the life of the strategy.

How it works. The user configures the strategy parameters and grants a delimited allowance. From that point, the contract logic executes operations when conditions are met. The user can halt the strategy and revoke the allowance at any time, without intermediaries.

Relevant detail. The available strategies cover distinct operating profiles: range accumulation, scheduled periodic purchase, and staged exit. Each has a visual representation rendering its state on the chart, so that the user sees where each level sits and what has occurred at each one, rather than relying on a table of figures.

Where it grows. Additional strategies, finer parameterisation, and a connection to the analysis layer so that configuration can begin from a signal rather than manual entry.

6.3 Peer-to-peer market · OPERATIONAL

What it is. A marketplace where two users agree directly on the exchange of an asset, with terms encoded in a contract acting as guarantor.

What it solves. Direct commerce between individuals demands trust between strangers, and the usual solutions either introduce a custodian holding the funds or leave the user exposed.

How it works. The seller deposits the asset into the contract, which holds it under conditions. The buyer settles payment through the channel both parties agreed. Upon confirmation, the contract releases the asset. Where a discrepancy arises, a resolution procedure exists. Neither can the seller disappear with the asset nor the buyer claim without having paid, because the conditions do not rest on either party's good faith.

Relevant detail. The market accepts any token on the network rather than a closed set of assets. Participants accumulate an operating history recorded on chain, constituting a verifiable reputation rather than a score awarded by a platform.

Where it grows. Broader payment methods, refinement of the dispute procedure, and more developed reputation tooling.

6.4 Staking · OPERATIONAL

What it is. The mechanism through which ecosystem activity is redistributed toward those who commit assets to the system.

What it solves. Reward mechanisms commonly depend on issuing a proprietary token, which produces a dilution dynamic. Here the reward derives from the real activity of the modules.

How it works. Each module generating economic activity allocates a proportion to the staking contract. That proportion is configurable and encoded. Users committing assets share in the redistribution.

Relevant detail. This is where the system's interconnection becomes tangible. Staking is not an isolated module: it is the common destination of every other module's activity. Each new service joining the ecosystem will connect here as well, which means the mechanism strengthens as the system grows.

Status. Operational and connected to the exchange and security modules. The connection to the derivatives module will be made once that module is complete.

6.5 Wallet security · OPERATIONAL

What it is. A suite of seven analysis and protection tools operating on any wallet on the network.

What it solves. A substantial share of losses in the sector stems not from technical breaches but from permissions granted and forgotten, from spoofed addresses users copy out of their own history, and from contracts signed without understanding what they authorise. These are losses avoidable through information, and that information sits on the chain but out of reach of the average user.

How it works. The tools read the on-chain activity of the wallet under analysis and present the result in interpretable form. Permission analysis locates every active allowance, classifies it by risk, identifies which protocols they correspond to, and allows revocation in a single operation. The contract checker examines an address before the user signs anything. Emergency evacuation moves all assets to a safe wallet in a guided sequence. The wallet watcher shows the complete composition and history of any address. The transaction verifier confirms whether an operation actually occurred, between which addresses and for what amount. The address poisoning detector locates spoofed addresses planted in the user's history that mimic real ones in order to induce a copying error. And the network cost fund allows a user without resources to obtain the minimum required to move their own assets.

Relevant detail. This module is the clearest illustration that the ecosystem is not an exchange with features bolted on. It is a complete security service that exists because the problem exists, and that sustains itself economically within the system.

Where it grows. Fraudulent token detection, contract verification against sale-blocking logic, and deeper risk analysis.

6.6 Derivatives · UNDER ACTIVE DEVELOPMENT

What it is. The module for trading futures contracts.

Status. The interface is built. The contract is under active development and is the project's immediate priority. Its completion will round out coverage of the standard trading profiles within the same non-custodial layer.

Why it matters architecturally. Derivatives trading is where most platforms abandon the non-custodial model, because margin and liquidation management is considerably simpler with custody. Solving it without custody is the most demanding technical challenge in the project.

6.7 Market and liquidity analysis layer · PARTIAL

What it is. An analysis layer presenting liquidity conditions, market structure and the relevant operating references.

Relevant detail. The design objective is to remove the distance between analysis and execution. In the conventional workflow, a trader analyses in one tool, decides, and executes on a separate platform, with the latency and friction that introduces. Here analysis and execution inhabit the same layer: a position can be opened from within the analysis itself. It is a decision born of experience operating in systems where that separation exists.

Status. Partially operational functions, under continuing development.

6.8 Education and prize pool · UNDER DEVELOPMENT

Status. Both modules have deployed contracts but are being rebuilt and are not ready for use. The education module additionally requires an external automation layer for tracking participant progression.

7. User flows

Access. The user opens the application. If their wallet has previously authorised the domain, the connection is re-established silently without prompting. If not, they connect explicitly. There is no registration, no credentials and no identity verification. The connected address is the identity.

Exchange. The user selects the assets and amount. The application queries liquidity and presents the estimated result. Where the source asset requires prior authorisation, it is requested as a separate and explicit operation. The user signs, and the transaction travels from their wallet to the network. The interface reflects confirmation once the chain produces it.

Automated strategy. The user defines the parameters. The application calculates the representation of the strategy and renders it on the chart before anything is activated. On confirmation, the bounded allowance is granted and the strategy becomes active. From then on it operates according to its conditions, and the user may halt it and revoke the allowance whenever they choose.

Peer-to-peer market. The seller publishes an offer and escrows the asset through a signed operation. The buyer accepts and settles payment through the agreed channel. On confirmation, the contract releases the asset. The entire sequence is recorded on chain.

Security analysis. The user opens the module, which reads the on-chain activity of the specified wallet and presents the result. Corrective actions, such as revoking a permission, are operations signed by the user themselves.

8. Infrastructure and dependencies

8.1 What the system depends on

Chain access is obtained through node providers, with redundancy across several. Historical data queries require an indexing service, since querying the chain directly to reconstruct complete histories is impractical. Price references come from specialist sources. The liquidity against which trades execute resides in third-party protocols.

8.2 A lesson worth documenting

During development, the indexed data service we relied on was discontinued by its provider. The effect was immediate: functions depending on historical queries stopped returning information, with no visible error, simply empty.

The resolution involved migrating to an alternative provider and, more importantly, redesigning the data access layer so that queries adapt to provider limits rather than assuming them. The episode left a conclusion that has shaped subsequent decisions: every external dependency should be treated as potentially transient, and the architecture must allow it to be replaced without redesigning what sits above it.

It is worth noting that failures of this kind affect the availability of information, never the funds, which remain in users' wallets regardless of which provider is operating.

8.3 Domain and automated classification

The current domain is temporary and corresponds to the development phase. The automated risk classification systems operating within wallets apply conservative criteria to domains without history, which has produced a classification currently under review. The definitive domain will be established at launch.

9. Security: surfaces, mitigations and limits

This chapter describes the system's real risk surfaces. It does not claim any of them is entirely resolved, because no connected system can claim that.

Custody. Eliminated by design. There is no risk of the platform losing funds it does not hold.

Token permissions. The most significant vector in non-custodial systems. Mitigation is twofold: the application requests bounded rather than unlimited allowances wherever the operation permits, and a dedicated module exists to locate, classify and revoke active permissions. Risk remains insofar as the user may grant allowances outside our platform.

Errors in contract code. The contracts apply established patterns: reentrancy protection, checks before effects, explicit access restrictions, a pinned compiler version. They have been internally reviewed. They have not undergone independent audit, which is anticipated on completion of development. This risk is real and we do not minimise it.

Upgrade authority. Whoever controls the owner addresses can modify contract logic. Current mitigation is that those addresses are multiple and intended to operate as cold storage with infrequent, controlled access. A natural evolution toward multi-signature schemes is anticipated. It is a trust vector that exists and that we prefer to declare rather than omit.

Impersonation and social engineering. No architecture protects a user who signs on a spoofed domain. Mitigation is educational and product-level: the security module exists precisely so that users can detect and reverse the consequences of an ill-advised signature.

External dependencies. A provider outage degrades the experience but does not compromise funds. Mitigation lies in redundancy and replaceability.

Price references. A compromised or stale oracle can produce incorrect calculations. Mitigation lies in source selection and range validation; the risk does not disappear.

10. Current state of the ecosystem

Operational. Asset exchange with token listing. Peer-to-peer market. Staking. Wallet security with its seven tools. Ten contracts deployed, verified and interconnected.

Under active development. The derivatives module, with a complete interface and contract in progress. Expanded exchange protocol support.

Under development. Education module. Prize pool. Market and liquidity analysis layer.

Anticipated. Complete administrative panel. Independent external audit. Cross-chain operation. Incorporation of further protocols and services into the shared accounting layer.

Not open to external users. The system has not been opened to the public and will not be until development reaches a state that warrants it. Current testing is conducted exclusively by the team.

11. Differentiation, with the evidence first

11.1 What we do not claim

Research into the current landscape shows the non-custodial model to be well established. Numerous platforms operate without custody, and several require no identity verification. It would be inaccurate to present either trait as distinguishing, and we do not.

11.2 What the evidence does show

Analysis of the sector reveals a consistent pattern: existing non-custodial platforms are **specialised**. One solves exchange. Another, derivatives. Another, peer-to-peer commerce. Those combining several functions tend to retain a hybrid component, keeping a centralised or custodial layer at the core, or segmenting the experience between a decentralised mode and one that is not. And geographic restrictions persist even at platforms presenting themselves as decentralised.

The defensible formulation, therefore, is not that we are first to operate without custody. It is that **the convergence of modules we are assembling under a single non-custodial layer, with no intermediate hybrid component, joined by an accounting layer that interconnects them, has no directly identifiable equivalent in the current landscape.**

11.3 Where the technical interest lies

The system is not a sum of functions. It is a set of modules that feed one another. Each service's activity converges on a shared accounting layer and from there feeds the rewards mechanism. Every new service incorporated increases the density of the whole, because it connects to the same structure. An ecosystem of this kind does not grow linearly by adding tabs to an interface: it gains internal coherence as it expands.

That is the objective ordering the technical decisions from the outset, and it is also why the proxy architecture is not an implementation detail but a condition: a system intended to incorporate modules over a period of years cannot be built on frozen contracts.

12. Strategic considerations and open questions

12.1 Where there is room to scale

The modular architecture allows services to be added without redesigning the core, and the accounting layer means each addition reinforces the whole. The marginal cost of adding a module is writing and connecting it, not rebuilding the system.

The structural absence of geographic restrictions widens potential reach to users excluded from conventional platforms by jurisdiction, a documented and non-trivial segment.

The security module has an uncommon characteristic: it addresses a real and quantified problem in the sector, and it works on any wallet, not only on ecosystem users.

12.2 Where greater maturity is required

Independent external audit is the most consequential outstanding step, and it is contingent on completing development. The derivatives module represents the most demanding technical challenge. Decentralisation of administrative control, today concentrated in owner addresses, has a natural evolution toward multi-signature schemes that has not yet been executed. And the system has not been subjected to the test of real use, which is the only one that reveals what no internal review finds.

12.3 Questions we leave open

A reader experienced in this sector likely holds a formed view on matters where ours is necessarily limited. Among them: what the right moment is to subject the system to external audit, and with what scope. What governance scheme for administrative control is reasonable for a system of this nature. What sequence of public opening minimises the risk of a compromised first impression. And which additional modules would have the greatest effect on the coherence of the whole.

These are questions we raise because they strike us as genuinely open, not because we have an answer prepared.

Appendix · Information pending confirmation

The following items are explicitly noted as unverified as at the date of this document: the explorer verification status of the contracts belonging to modules under reconstruction; the definitive development completion schedule, subject to team availability; and the project's final domain, not yet established.

Working document. The architecture described is under active development and its components may evolve. Contract addresses correspond to the deployments current as at the date indicated.